

Fashion-MNIST — CNN e Transfer Learning

Documentação completa do projeto: dataset, notebook de treino, modelos, app de inferência em Streamlit, tracking de experimentos com MLflow e orquestração com Airflow, tudo containerizado com Docker.

Sumário

- [Visão geral](#)
- [Estrutura do projeto](#)
- [Dataset](#)
- [Notebook de treino](#)
- [Modelos e resultados](#)
- [App Streamlit](#)
- [Capturas de tela do deploy](#)
- [Stack de infraestrutura \(Docker + MLflow + Airflow\)](#)
- [Rodando localmente sem Docker](#)
- [Troubleshooting](#)

Visão geral

O projeto classifica imagens do dataset **Fashion-MNIST** (10 categorias de roupas e calçados, 28x28 pixels, escala de cinza) usando:

1. Uma **CNN construída do zero** (baseline), treinada diretamente em 28x28x1.
2. Cinco modelos de **Transfer Learning** (`MobileNetV2` , `VGG16` , `VGG19` , `ResNet50` , `EfficientNetB0`), usando backbones pré-treinados na ImageNet com fine-tuning.

Os modelos treinados são servidos através de um **app Streamlit** para inferência interativa, e suas métricas/artefatos podem ser importados para o **MLflow** (via um DAG do **Airflow** ou diretamente por script), com tudo orquestrado por **Docker Compose**.

Estrutura do projeto

```
.
├── airflow/
│   ├── dags/
│   │   └── log_fashion_mnist_models_to_mlflow_dag.py    # DAG que importa métricas/artefatos
│   └── logs/
├── app/
│   └── app.py                                             # App Streamlit de inferência
├── doc/
│   └── README.md                                         # Este arquivo
├── docker/
│   ├── Dockerfile.streamlit                             # Imagem do app Streamlit
│   ├── Dockerfile.mlflow                               # Imagem do servidor MLflow
│   └── requirements-mlflow.txt
├── docker-compose.yml                                   # Orquestra streamlit + mlflow + airflow
├── input/                                                # Dataset Fashion-MNIST (CSV) baixado do Kaggle
├── mlflow/                                               # Dados do MLflow (sqlite + artifacts) – volume persistente
├── models/                                               # Modelos treinados (.keras) e métricas/históricos (.pkl)
├── notebook/
│   └── Fashion_MNIST_1__CNN_Transfer_Learning_.ipynb    # Notebook de treino (EDA, treino)
├── scripts/
│   └── log_models_to_mlflow.py                          # Script que registra os modelos já treinados no MLflow
├── src/
│   └── download_dataset.py                              # Baixa o dataset do Kaggle e carrega num SQLite local
├── sql/
│   └── fashion_mnist.db                                  # Dataset também disponível como SQLite (gerado pelo notebook)
├── output/                                              # Imagens/gráficos exportados do notebook
└── requirements.txt                                     # Dependências do app Streamlit
```

Dataset

Fashion-MNIST — 70.000 imagens em escala de cinza (28x28), 10 classes:

Label	Classe
0	T-shirt/top
1	Trouser

Label	Classe
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

Baixando o dataset

O script `src/download_dataset.py` : 1. Lê as credenciais do Kaggle (`KAGGLE_USERNAME` , `KAGGLE_KEY`) de um arquivo `.env` na raiz do projeto (não versionado). 2. Baixa o dataset via `kagglehub` e copia os CSVs (`fashion-mnist_train.csv` , `fashion-mnist_test.csv`) para `input/` . 3. Carrega os CSVs num banco SQLite em `sql/fashion_mnist.db` (tabelas `fashion_mnist_train` e `fashion_mnist_test` , com índice na coluna `label`).

```
python src/download_dataset.py
```

Precisa ser rodado num terminal Windows local (o script aborta se detectar outro SO, pois foi pensado para esse ambiente específico).

Notebook de treino

`notebook/Fashion_MNIST_1__CNN_Transfer_Learning_.ipynb` contém o pipeline completo:

1. **EDA**: distribuição de classes, amostras por classe, imagem média por classe.
2. **Pré-processamento**: pixels normalizados para `[0, 1]` , reshape para `(28, 28, 1)` .
3. **CNN baseline**: arquitetura própria (3 blocos convolucionais + BatchNorm + Dropout + Dense), 50 épocas.

4. **Transfer Learning:** para cada backbone (`MobileNetV2` , `VGG16` , `VGG19` , `ResNet50` , `EfficientNetB0`): - Resize de 28x28 para 96x96 e conversão de 1 para 3 canais (grayscale → RGB), **embutidos no próprio grafo do modelo** via camadas `Resizing` / `Concatenate` . - Pré-processamento específico de cada backbone (`[-1, 1]` para `MobileNetV2`, `caffe` para `VGG/ResNet`, `[0, 255]` para `EfficientNet`) também embutido como camadas do modelo. - Treino em duas fases: backbone congelado (8 épocas) e fine-tuning com as últimas 30 camadas descongeladas (5 épocas).
5. **Avaliação:** acurácia, F1 macro, matriz de confusão, curva ROC/AUC por classe, classification report, análise dos erros mais confiantes.

Importante: como o pré-processamento (resize, RGB, normalização) está embutido no `.keras` salvo, qualquer código de inferência só precisa fornecer a imagem original em (28, 28, 1) , valores em `[0, 1]` — é assim que o `app/app.py` e o `scripts/log_models_to_mlflow.py` tratam os modelos.

Modelos e resultados

Modelos salvos em `models/*.keras` , métricas em `models/results.json` , históricos de treino em `models/cnn_baseline_history.json` (baseline) e `models/tl_histories.json` (transfer learning, por fase `frozen` / `finetune`).

Modelo	Acurácia (teste)	F1 macro	AUC macro	Parâmetros	Tempo de treino
CNN baseline	94.21%	0.9419	0.9974	288.618	—
VGG19	92.38%	0.9233	0.9955	20.029.514	1.190,87 s
VGG16	92.26%	0.9220	0.9953	14.719.818	956,50 s
ResNet50	92.05%	0.9199	0.9953	23.608.202	492,24 s
MobileNetV2	91.65%	0.9163	0.9951	2.270.794	185,15 s
EfficientNetB0	90.80%	0.9078	0.9942	4.062.381	235,32 s

A CNN baseline, mesmo muito menor, supera todos os modelos de transfer learning em acurácia, F1 macro e AUC macro — esperado, já que os backbones foram pré-treinados em imagens naturais (ImageNet) e aqui precisam lidar com imagens pequenas, em escala de cinza, upscaladas artificialmente. O AUC macro muito alto em todos os modelos (≥ 0.994) mostra que mesmo os

modelos com menor acurácia separam bem as classes em termos de ranking de probabilidade — os erros se concentram em poucas classes visualmente parecidas (ex.: Shirt vs. T-shirt/top vs. Pullover vs. Coat), não em confusões aleatórias.

App Streamlit

`app/app.py` permite escolher qualquer um dos 6 modelos, enviar uma imagem (upload) e ver a classe prevista com o gráfico de probabilidade por classe.

- Pré-processamento: a imagem enviada é convertida para escala de cinza, redimensionada para 28x28 e normalizada para `[0, 1]` — igual ao que o modelo espera. Há um checkbox para inverter cores (fotos reais costumam ter peça escura sobre fundo claro; o dataset original é o inverso).
- Métricas de cada modelo (acurácia/F1) são exibidas na barra lateral, lidas de `models/results.json`.

Rodando localmente (fora do Docker)

Requer um ambiente Python com TensorFlow funcional (ver [Troubleshooting](#) sobre o ambiente `fashion-cnn-app`):

```
streamlit run app/app.py
```

Abre em `http://localhost:8501`.

Capturas de tela do deploy

App Streamlit rodando em `http://localhost:8501`, classificando imagens de teste em tempo real:

Exemplo 1 — Ankle boot (VGG19, 99.1% de confiança)

Modelo

Escolha o modelo

VGG19 (Transfer Learning) ▾

Acurácia (teste)

92.53%

Macro F1

0.9248

Parâmetros: 20,029,514

☐ Inverter cores ⓘ

Fashion-MNIST — CNN & Transfer Learning

Envie uma foto de uma peça de roupa/calçado (fundo simples, de preferência centralizada) e escolha um modelo treinado para classificá-la.

Imagem da peça



Drag and drop file here

Limit 200MB per file • PNG, JPG, JPEG

Browse files



1007.png 401.0B



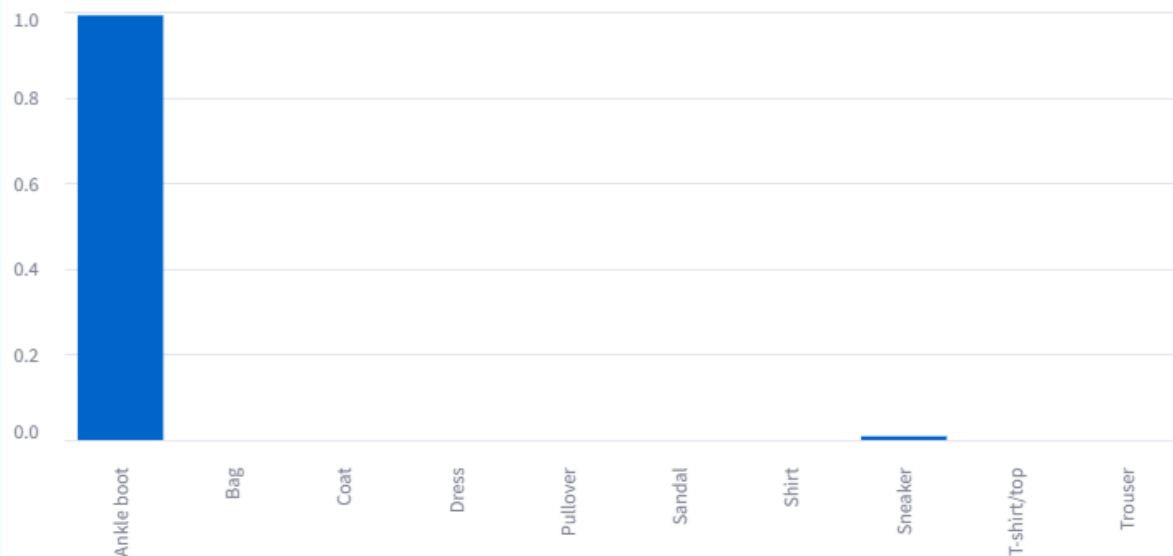
Imagem enviada



Entrada pré-processada (28x28)

Previsão: **Ankle boot** (99.1% de confiança)

Probabilidade por classe



Exemplo 2 — Sneaker (100% de confiança)



Fashion-MNIST — CNN & Transfer Learning [↗](#)

Envie uma foto de uma peça de roupa/calçado (fundo simples, de preferência centralizada) e escolha um modelo treinado para classificá-la.

Imagem da peça



Drag and drop file here

Limit 200MB per file • PNG, JPG, JPEG

Browse files



102.png 342.0B



Imagem enviada



Entrada pré-processada (28x28)



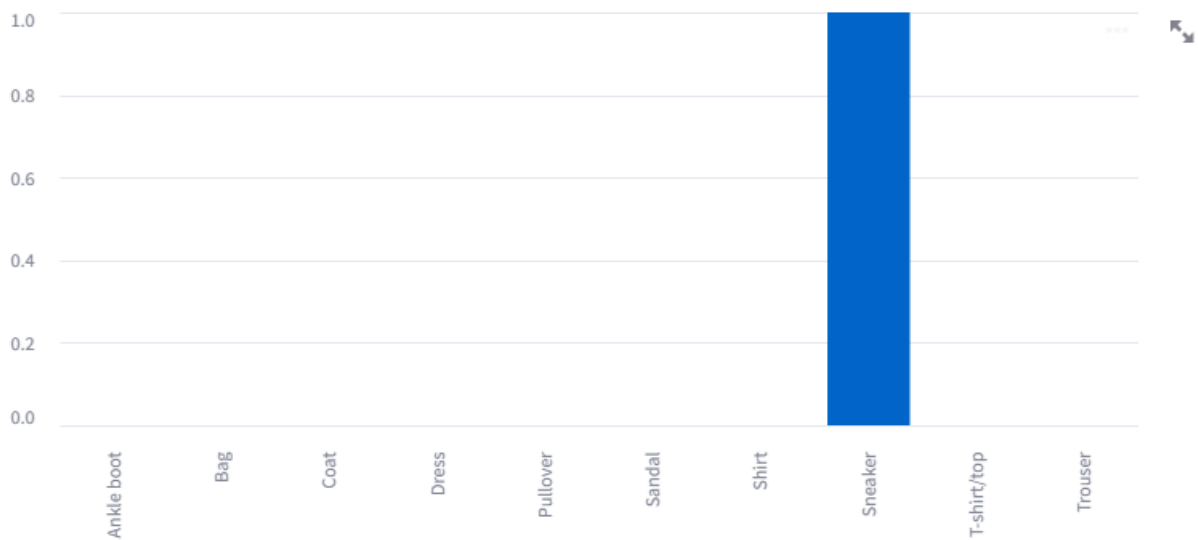
Imagem enviada



Entrada pré-processada (28x28)

Previsão: **Sneaker** (100.0% de confiança)

Probabilidade por classe



Stack de infraestrutura (Docker + MLflow + Airflow)

Orquestrada via `docker-compose.yml` , com três serviços:

Serviço	Porta no host	Descrição
streamlit	8501	App de inferência (build: <code>docker/Dockerfile.streamlit</code>)
mlflow	5000	Servidor de tracking (build: <code>docker/Dockerfile.mlflow</code>), backend SQLite + artifacts locais em <code>./mlflow</code>
airflow	8081	<code>apache/airflow:2.9.3-python3.11</code> rodando em modo <code>standalone</code> (webserver + scheduler + triggerer num único container)

Subindo tudo

```
docker compose up -d --build
```

- Streamlit: `http://localhost:8501`
- MLflow UI: `http://localhost:5000`
- Airflow UI: `http://localhost:8081` (usuário `admin` ; a senha é gerada automaticamente a cada start e fica em `standalone_admin_password.txt` dentro do container — veja abaixo como pegá-la)

```
docker compose exec airflow cat /opt/airflow/standalone_admin_password.txt
```

O Airflow usa `SequentialExecutor` + SQLite **sem volume persistente de metadados** — a cada restart do container ele reinicializa do zero (DAGs continuam lá, pois `airflow/dags` é montado como volume; mas histórico de runs e a senha de admin mudam). Suficiente para o escopo deste projeto; para produção, trocar por `LocalExecutor` / `CeleryExecutor` + Postgres.

Importando métricas e modelos para o MLflow

O `scripts/log_models_to_mlflow.py` lê `models/results.json` , `models/cnn_baseline_history.json` e `models/tl_histories.json` , e para cada modelo cria uma run no MLflow com: - `params` : tamanho de imagem de entrada, número de parâmetros. - `metrics` : `test_accuracy` , `test_macro_f1` , `train_time_s` , e as curvas de treino (`accuracy` , `loss` , `val_accuracy` , `val_loss`) por época. - `artifact` : o arquivo `.keras` correspondente.

Não retreina nada — apenas importa o que já foi treinado no notebook.

Via Airflow (recomendado): dispare a DAG `log_fashion_mnist_models_to_mlflow` pela UI (<http://localhost:8081>) sempre que os modelos/métricas mudarem.

Rodando o script manualmente de dentro de um container (necessário para o caminho de artifacts `/mlflow-data` resolver corretamente):

```
docker compose exec airflow python /opt/airflow/scripts/log_models_to_mlflow.py
```

⚠ Não rode `scripts/log_models_to_mlflow.py` diretamente do Windows/host apontando para `http://localhost:5000` esperando que os artifacts caiam no volume do Docker — o MLflow client escreve artifacts direto no filesystem do caminho retornado pelo servidor (`/mlflow-data/...`), que só existe dentro dos containers. Rodado no host, o Python cria sem querer uma pasta `C:\mlflow-data\...` solta no disco. Sempre rode o script de dentro de um container que tenha `./mlflow` montado em `/mlflow-data` (o serviço `airflow` já está configurado assim).

Arquitetura do fluxo de dados

```
notebook (treino) → models/*.keras + *.json
    |
    +--- app/app.py (Streamlit)
    |      serve inferência
    |
    +--- scripts/log_models_to_mlflow.py
         (disparado via Airflow DAG)
         |
         +--- MLflow Tracking Server
              (métricas, curvas, artifacts)
```

Rodando localmente sem Docker

`requirements.txt` (raiz) lista as dependências do app Streamlit: `streamlit`, `tensorflow==2.17.0`, `numpy<2`, `pillow<11`.

```
pip install -r requirements.txt
streamlit run app/app.py
```

Troubleshooting

`ImportError: DLL load failed` ao importar `tensorflow` no Windows

Esse projeto rodou inicialmente num ambiente Anaconda `base` com `tensorflow==2.20.0` que falhava ao carregar a DLL nativa (`_pywrap_tensorflow_common.dll`, ~944 MB) — causa raiz não identificada com certeza (suspeita de antivírus/Core Isolation interferindo), **não** era corrupção do pacote.

Solução aplicada: criar um ambiente conda isolado dedicado a este projeto, com versões mais antigas e testadas:

```
conda create -n fashion-cnn-app python=3.11 -y
conda activate fashion-cnn-app
pip install tensorflow==2.17.0 streamlit==1.36.0 "numpy<2" "pillow<11"
```

O ambiente `base` do Anaconda **não deve** ser usado para rodar este projeto — ele compartilha dependências com outros projetos (`mlflow`, `gradio`, `numba`, `scikit-learn` etc.) e teve `numpy` / `protobuf` / `pillow` atualizados durante esse diagnóstico, o que pode ter quebrado compatibilidade com eles.

`st.image()` got an unexpected keyword argument 'use_container_width'

O parâmetro `use_container_width` só existe em versões do Streamlit mais recentes que `1.36.0`. Usar `use_column_width=True` no lugar (já corrigido em `app/app.py`).

Porta já em uso (8080 para o Airflow)

Se já existir outro Airflow rodando na máquina usando a porta `8080`, o serviço `airflow` deste projeto está mapeado para `8081:8080` no `docker-compose.yml` — ajuste se precisar.

Permissão negada ao logar artifacts no MLflow a partir do Airflow

O diretório `./mlflow/artifacts` é criado pelo container do `mlflow` (usuário `root`, permissão `755`). O container do `airflow` roda como uid `50000` / gid `0`. Se aparecer `PermissionError: [Errno 13] Permission denied: '/mlflow-data/...'`, rode:

```
docker compose exec mlflow chmod -R 775 /mlflow-data
```